Labo 11 — Labo pratique : premier contact avec le cluster

Objectif : installer k3s dans votre WSL, explorer le cluster avec `kubectl`, lancer un premier Pod, faire entrer l'image `api:1.0` construite avec Podman, puis voir de vos yeux la boucle de réconciliation — en passant par la passerelle `podman generate kube`.

Prérequis — Partie Docker terminée (labos 00 à 10). Image `api:1.0` présente dans Podman, `~/labo-docker/07/nginx-pod.conf` présent. Ubuntu/WSL avec `systemd=true`, `sudo`, et au moins 4 Go de RAM pour WSL. Les sorties ci-dessous viennent de k3s v1.34.1 ; le nom du nœud est celui de votre machine (ici `wsl-ubuntu`).

Fichiers fournis (`files/`) - `pod-api.yaml` — un Pod minimal pour l'API du fil rouge.

Étape 0 — Installer k3s (WSL uniquement)

```
curl -sfL https://get.k3s.io | sudo sh -s - --write-kubeconfig-mode 644
sudo systemctl status k3s --no-pager | head -3
which kubectl ; ls -l $(which kubectl)
```

Observez le script qui télécharge le binaire, crée le service `k3s.service` et le démarre (`active (running)`), et un `kubectl` qui n'est qu'un lien vers `/usr/local/bin/k3s`. Cette étape ne peut pas être rejouée dans le bac à sable de rédaction : les sorties sont décrites, pas citées. Comptez une minute avant que le nœud soit prêt.

Étape 1 — Faire connaissance avec le cluster

```
kubectl get nodes
kubectl get pods -A
kubectl cluster-info | head -2
ls -l /etc/rancher/k3s/k3s.yaml ; grep -E 'server:' /etc/rancher/k3s/k3s.yaml
kubectl config get-contexts
```

Observez un nœud `wsl-ubuntu Ready control-plane … v1.34.1+k3s1` ; dans `kube-system`, les composants fournis par k3s : `coredns`, `local-path-provisioner`, `metrics-server`, `traefik`, `svclb-traefik-…`, et deux Pods `helm-install-traefik…` en `Completed` ; l'API server sur `https://127.0.0.1:6443` ; un kubeconfig en `-rw-r--r--` ; et un contexte `default` sans namespace.

Explication. Un seul nœud joue le control plane et le nœud de travail. Tout ce que vous verrez passe par l'API server du `cluster-info`.

```
kubectl api-resources | head -8
kubectl explain pod.spec.containers | head -12
```

Observez la table des types d'objets (`configmaps`, `events`, `namespaces`…, avec leurs raccourcis et leur caractère `NAMESPACED`) et la documentation intégrée du champ `containers` d'un Pod.

Étape 2 — Un namespace pour le labo

```
kubectl create namespace lab11
kubectl config set-context --current --namespace=lab11
kubectl config view --minify -o jsonpath='{.contexts[0].context.namespace}' ; echo
```

Observez `namespace/lab11 created`, `Context "default" modified.`, puis `lab11` : toutes les commandes qui suivent travaillent dans ce namespace sans `-n`.

Étape 3 — Un premier Pod, sans fichier

```
kubectl run web --image=docker.io/library/nginx:alpine
kubectl get pods
sleep 8 ; kubectl get pods -o wide
kubectl describe pod web | sed -n '/^Events/,$p'
```

Observez `pod/web created`, un état `ContainerCreating` puis `Running` avec une IP `10.42.0.x` sur le nœud `wsl-ubuntu`, et la chronologie des Events : `Scheduled` (par `default-scheduler`), `Pulling`, `Pulled … in 3.75s`, `Created`, `Started` (par `kubelet`).

```
kubectl logs web | tail -2
kubectl exec web -- nginx -v
kubectl port-forward pod/web 18080:80 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080 | head -4 ; kill %1
kubectl delete pod web
```

Observez les logs de nginx, `nginx version: nginx/1.x`, la page d'accueil via le tunnel `port-forward` (aucun port publié : `kubectl` relaie), puis `pod "web" deleted from lab11 namespace`.

Explication. `logs`, `exec`, `delete` sont les gestes du labo 03 ; `describe` et ses Events sont nouveaux : le cluster journalise chaque étape de chaque composant.

Étape 4 — La passerelle Podman

```
mkdir -p ~/labo-docker/11 && cd ~/labo-docker/11
podman pod create --name demo -p 18090:80
podman run -d --pod demo --name demo-api api:1.0
podman run -d --pod demo --name demo-web -v ~/labo-docker/07/nginx-
pod.conf:/etc/nginx/conf.d/default.conf:ro nginx:alpine
sleep 2 ; curl -s localhost:18090/api/actuator/health ; echo
podman generate kube demo > demo-pod.yaml
grep -E 'kind:|name:|image:|hostPort|hostPath' demo-pod.yaml
podman pod rm -f -t 0 demo
podman kube play demo-pod.yaml | tail -3
sleep 3 ; curl -s localhost:18090/api/actuator/health ; echo
podman kube down demo-pod.yaml | tail -1 ; podman pod ps
```

Observez `{"status":"UP"}` depuis le pod Podman ; dans le YAML généré, `kind: Pod`, les deux `image:`, un `hostPort: 18090` et un volume `hostPath` vers votre fichier nginx ; après `kube play`, le pod recréé **depuis le YAML** (`demo-demo-api`, `demo-demo-web`) qui répond de nouveau ; puis plus aucun pod après `kube down`.

Explication. Podman écrit et lit le YAML de Kubernetes. Ce fichier décrit exactement votre pod — mais `hostPath` et `hostPort` le lient à votre machine ; un cluster les remplacera par une ConfigMap et un Service.

Étape 5 — L'image qui manque, puis l'import

```
kubectl apply -f <chemin-du-labo>/files/pod-api.yaml
sleep 6 ; kubectl get pods
kubectl describe pod api | sed -n '/^Events/,$p' | tail -4
```

Observez `pod/api created`, puis `ErrImagePull` ou `ImagePullBackOff`, avec dans les Events `Failed to pull image "localhost/api:1.0": … failed to pull and unpack image` et `Back-off pulling image`.

```
podman save localhost/api:1.0 | sudo k3s ctr images import -
sudo k3s crictl images | grep -E 'api|nginx'
sleep 20 ; kubectl get pods -o wide
```

Observez `Importing … done` côté import ; dans `crictl images`, `localhost/api 1.0` à côté de `docker.io/library/nginx alpine` (tirée par le cluster à l'étape 3) ; puis, sans rien relancer, `api 1/1 Running` avec son IP.

Explication. Le cluster a son propre magasin d'images. Le kubelet, qui réessayait en boucle, a trouvé l'image après l'import — `imagePullPolicy: IfNotPresent` lui dit d'utiliser ce qui est présent.

```
kubectl get pod api -o yaml | grep -E '^  phase:|podIP:|hostIP:'
kubectl exec api -- wget -qO- http://localhost:8080/ ; echo
kubectl port-forward pod/api 18080:8080 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/actuator/health ; echo ; kill %1
sudo k3s crictl ps | grep -E 'api|CONTAINER' | cut -c1-100
```

Observez `phase: Running`, un `podIP` en `10.42.0.x` et un `hostIP` (celui de votre WSL) ; `{"message":"bonjour depuis kubernetes","profile":"default"}` (la variable du manifeste) ; `{"status":"UP"}` par le tunnel ; et dans `crictl ps` un conteneur `api` — celui que `podman ps` ne montre pas.

Étape 6 — La boucle de réconciliation

```
kubectl delete pod api
kubectl get pods
kubectl create deployment api --image=localhost/api:1.0
sleep 6 ; kubectl get deployment,replicaset,pods
```

Observez le Pod seul supprimé pour de bon (`No resources found`), puis trois objets : `deployment.apps/api 1/1`, `replicaset.apps/api-5f68dd6795 1 1 1`, et un Pod `api-5f68dd6795-xxxxx Running`.

```
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $POD
sleep 6 ; kubectl get pods
kubectl get deployment api -o jsonpath='{.spec.replicas} {.status.readyReplicas}' ; echo
```

Observez `pod "api-5f68dd6795-xxxxx" deleted`, et aussitôt un **nouveau** Pod `api-5f68dd6795-yyyyy Running` ; `1 1` : un réplica voulu, un réplica prêt.

Explication. Vous avez supprimé le Pod ; le ReplicaSet a constaté l'écart et en a créé un autre, sans que vous ne demandiez rien. C'est le modèle déclaratif à l'œuvre — et le sujet du labo 12.

Nettoyage

```
kubectl delete namespace lab11
kubectl config set-context --current --namespace=default
kubectl get pods -A | grep -c lab11
```

Observez `namespace "lab11" deleted` (avec tout ce qu'il contenait : Deployment, ReplicaSet, Pod), le contexte revenu sur `default`, et `0`. L'image `localhost/api:1.0` reste dans le cluster, les labos suivants s'en servent. Pour arrêter k3s entre deux sessions : `sudo systemctl stop k3s` ; pour tout retirer un jour : `/usr/local/bin/k3s-uninstall.sh`.

Ce que vous devez pouvoir affirmer maintenant

- Un nœud `Ready`, six composants dans `kube-system`, un API server sur `127.0.0.1:6443`, un kubeconfig en `644`.
- `kubectl run` → `Scheduled`, `Pulling`, `Pulled`, `Created`, `Started` ; `logs`, `exec`, `port-forward`, `delete` comme avec Podman.
- `podman generate kube` écrit un `kind: Pod` avec `hostPort` et `hostPath` ; `podman kube play` le rejoue.
- Image Podman ≠ image du cluster : `ImagePullBackOff`, puis `podman save | sudo k3s ctr images import -`, et `Running` sans rien relancer.
- Un Pod seul supprimé ne revient pas ; sous un Deployment, un nouveau Pod apparaît en quelques secondes : `1 1`.
- `kubectl delete namespace lab11` emporte tout ce que le labo a créé.